

Four agents build a 4D tic-tac-toe game

A harness-and-model benchmark, with behavior analysis

2026-08-26

1 The experiment

Four harness/model combinations were each given the identical task: build a complete, polished, browser-based 4D tic-tac-toe game on a $3 \times 3 \times 3 \times 3$ board, in Python, end to end. Same `TASK.md`, same baseline commit, same definition of done. Every run was recorded wall-clock and per-API-call; the transcripts were then analyzed deterministically and by a second model reading the agent's own thinking. This report is the result.

2 Raw outcome

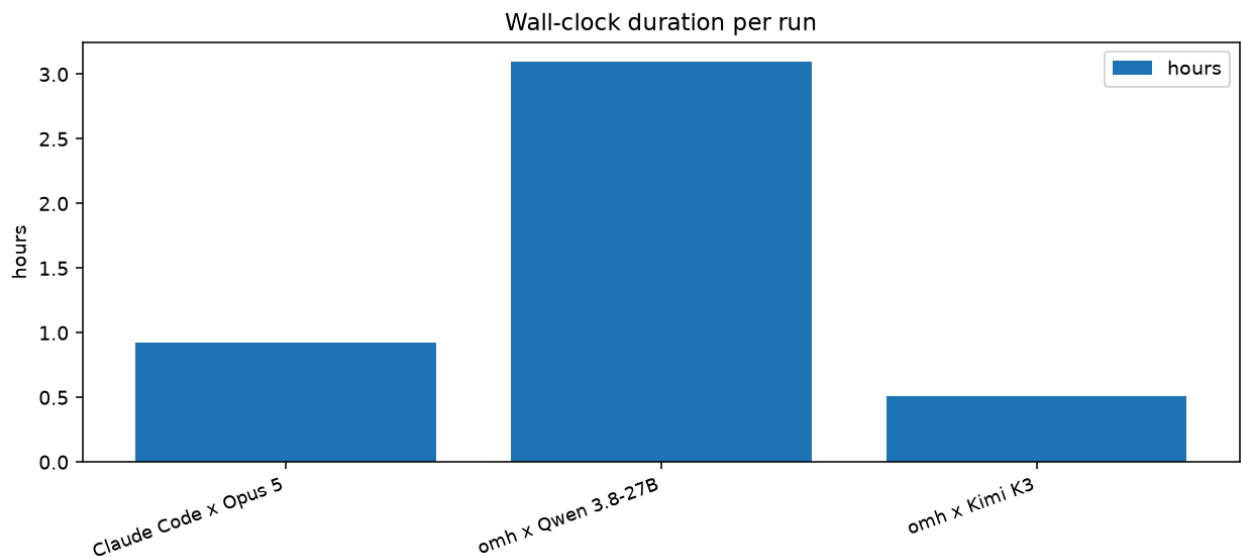


Figure 1: Wall-clock duration. Opus was dramatically fastest.

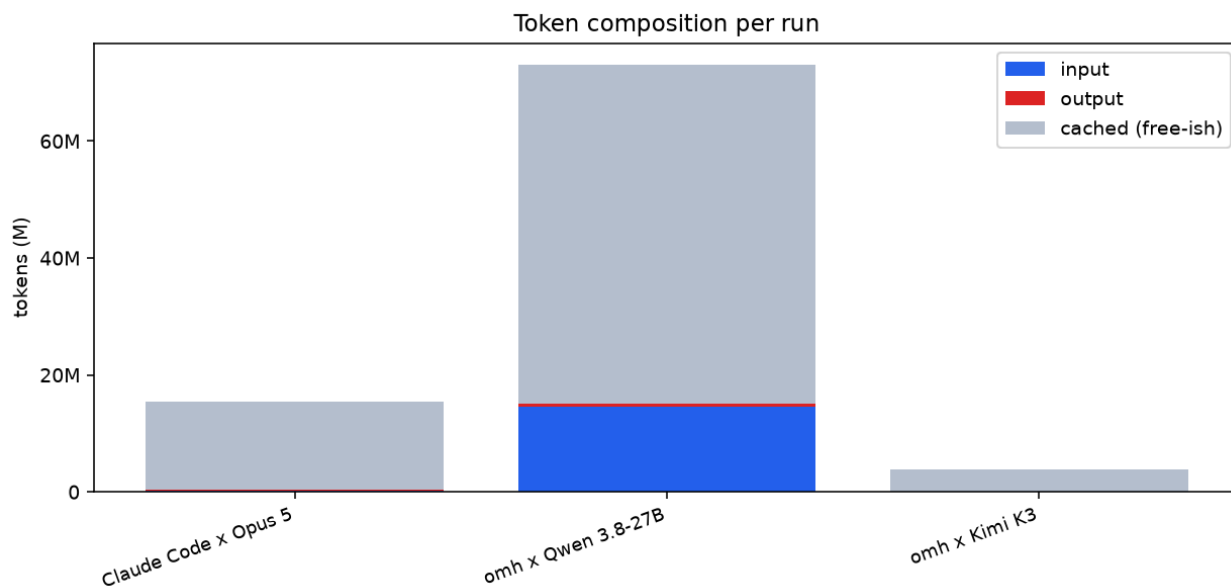


Figure 2: Token composition. Opus used the fewest total tokens; Qwen 3.8-27B used the most.

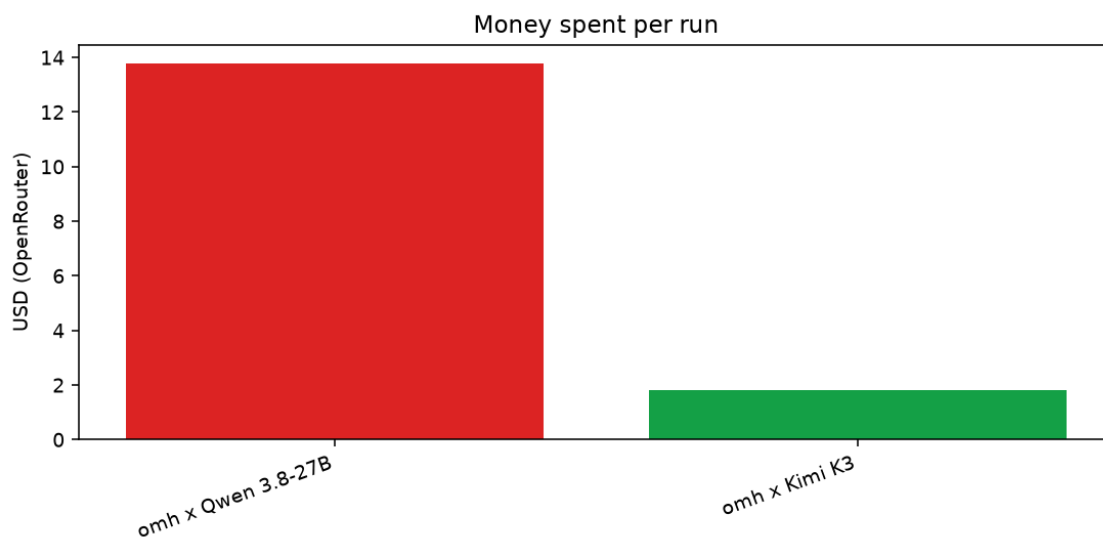


Figure 3: Direct cost (OpenRouter models only; Claude runs on a flat subscription).

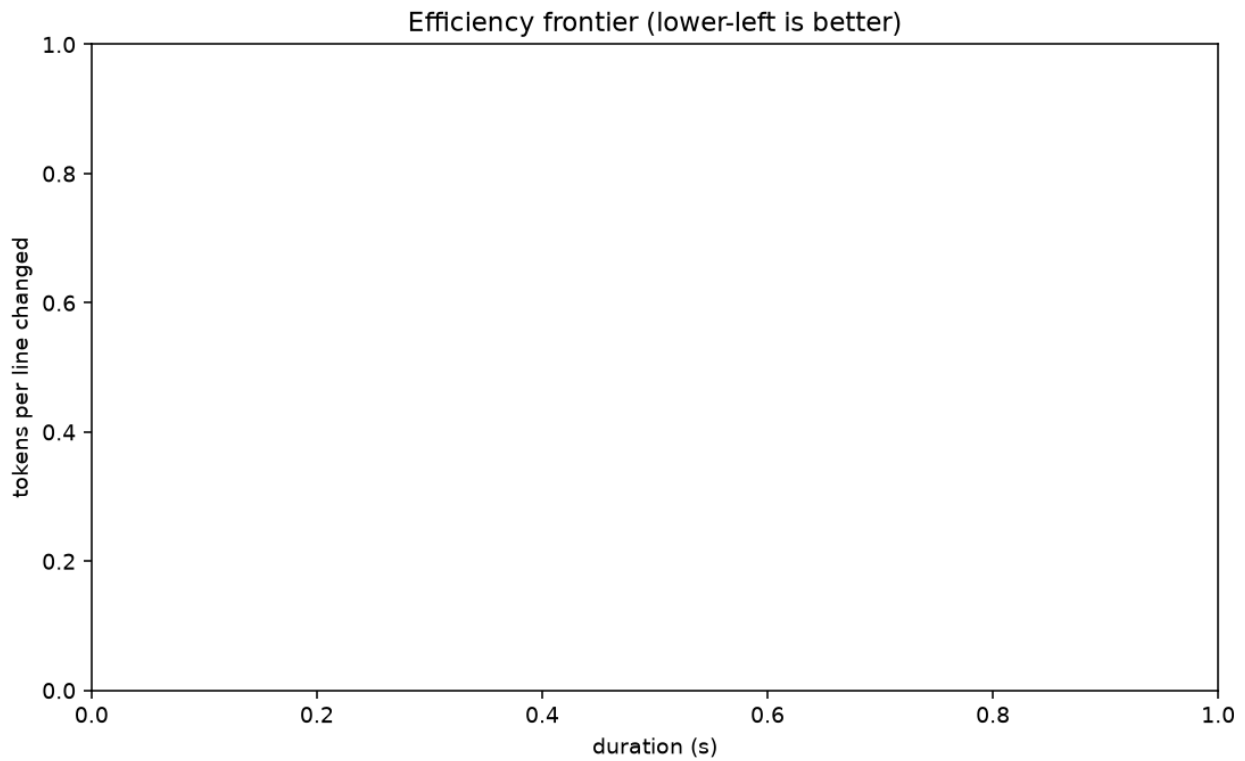


Figure 4: Efficiency frontier: time against tokens per line of final code. Lower-left is better.

3 Behavior under the hood

A smaller model read each transcript and scored each run on six dimensions. Figures below are the deterministic side: what the transcripts actually contain.

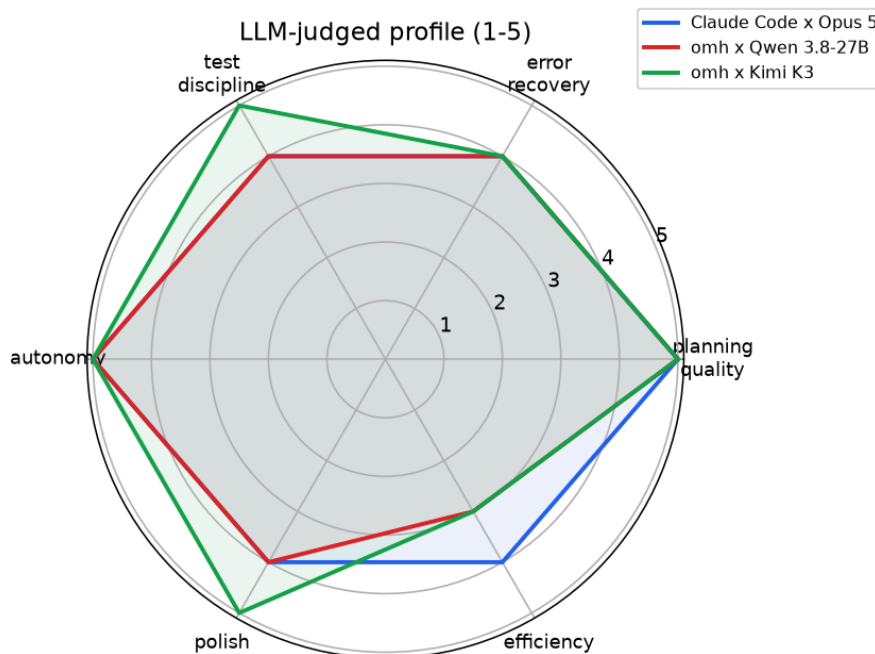


Figure 5: LLM-judged profiles across planning, recovery, discipline, autonomy, polish, efficiency.

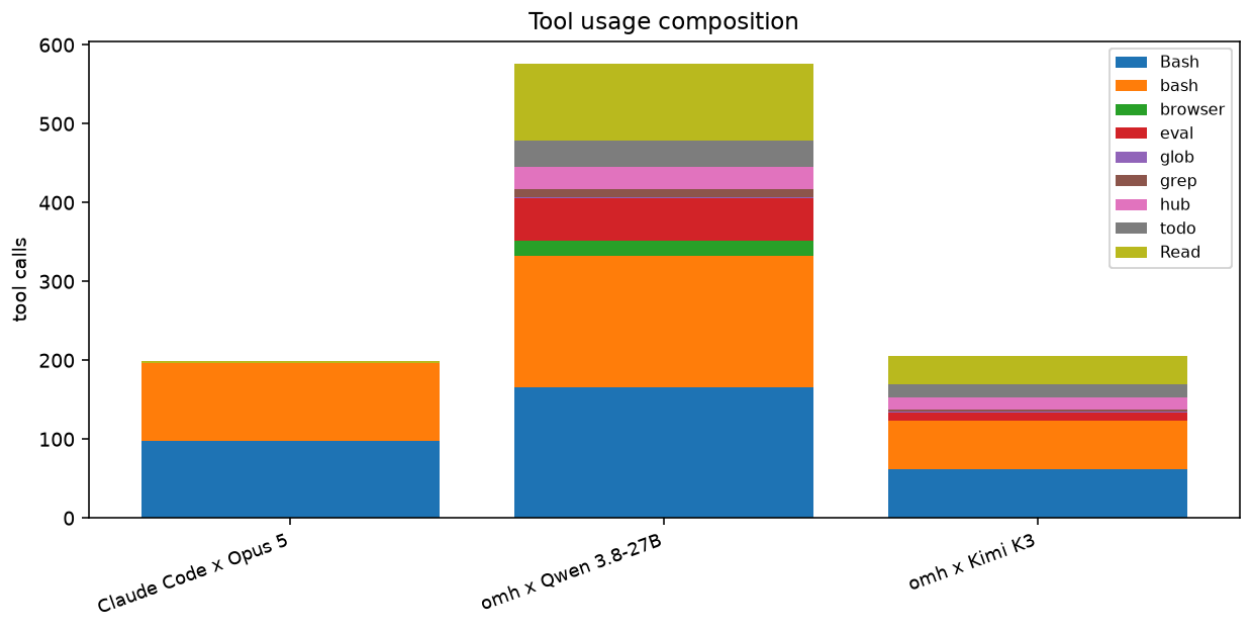


Figure 6: Tool usage composition. Bash-dominated for Opus; write/edit-heavy for the omh runs.



Figure 7: Test discipline against tool error rate.

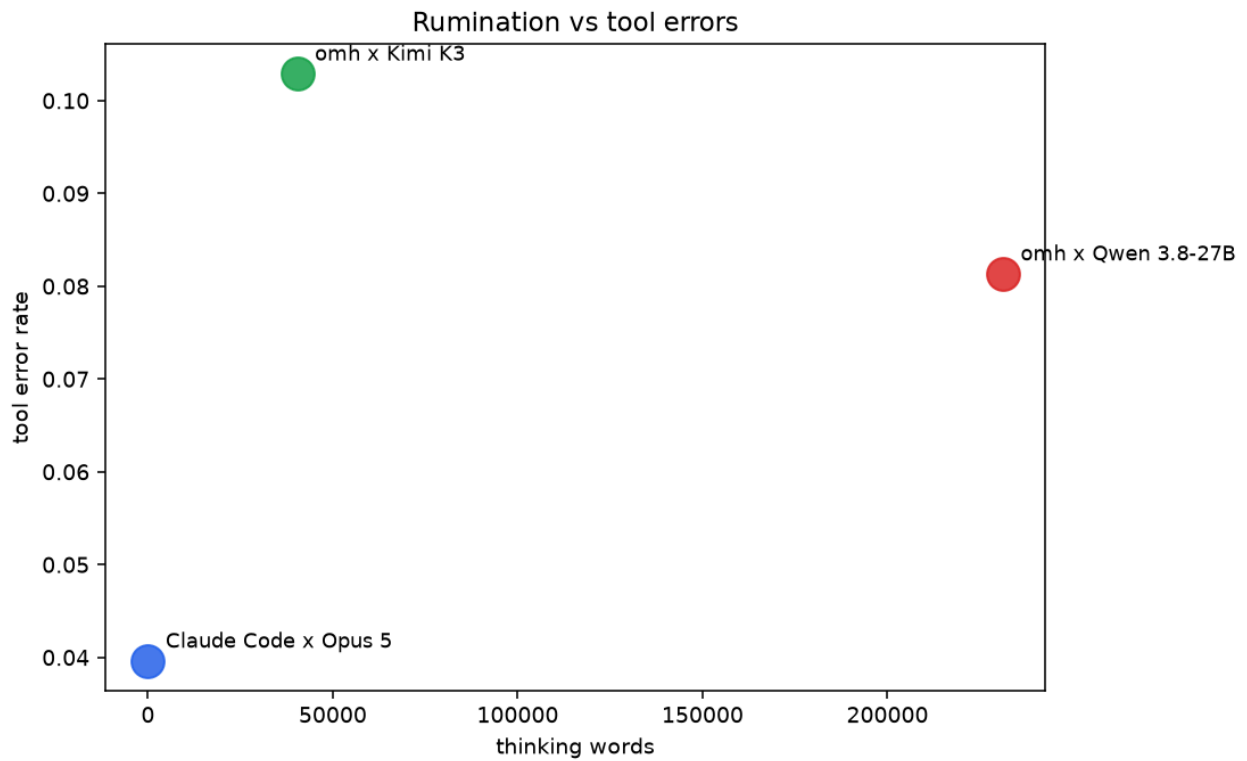


Figure 8: Thinking volume against tool errors. The two Kimi/GLM runs sit in the same quadrant.

4 How they talked to themselves

The agents' private reasoning, sampled and counted.

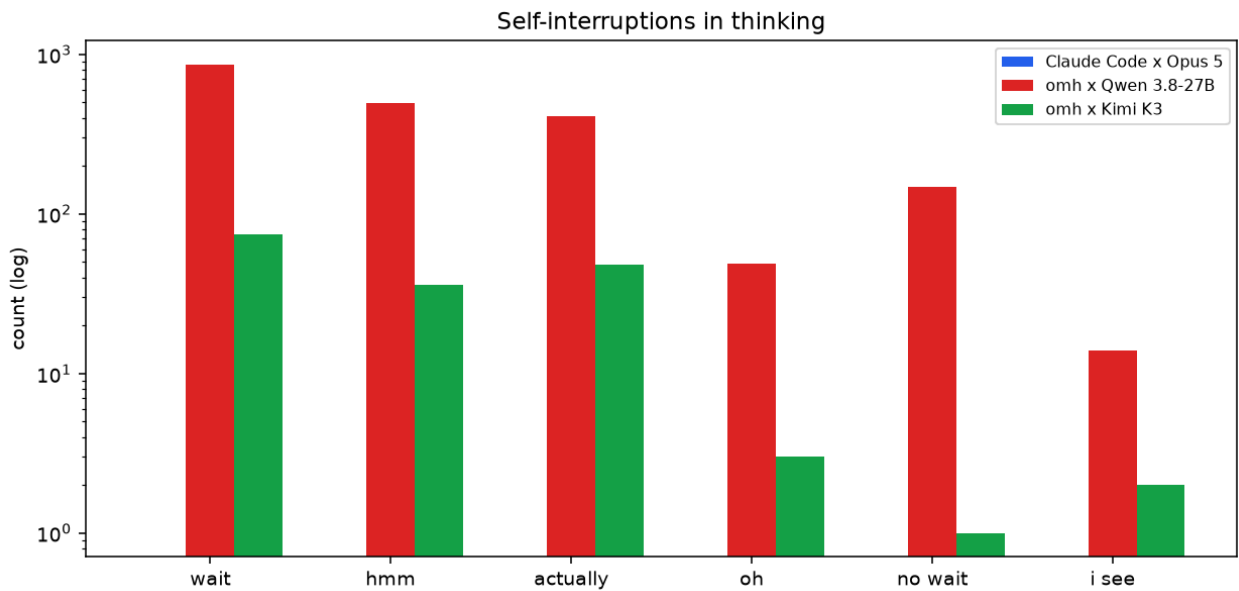


Figure 9: Self-interruptions in thinking (log scale). Qwen 3.8-27B said "wait" 867 times.

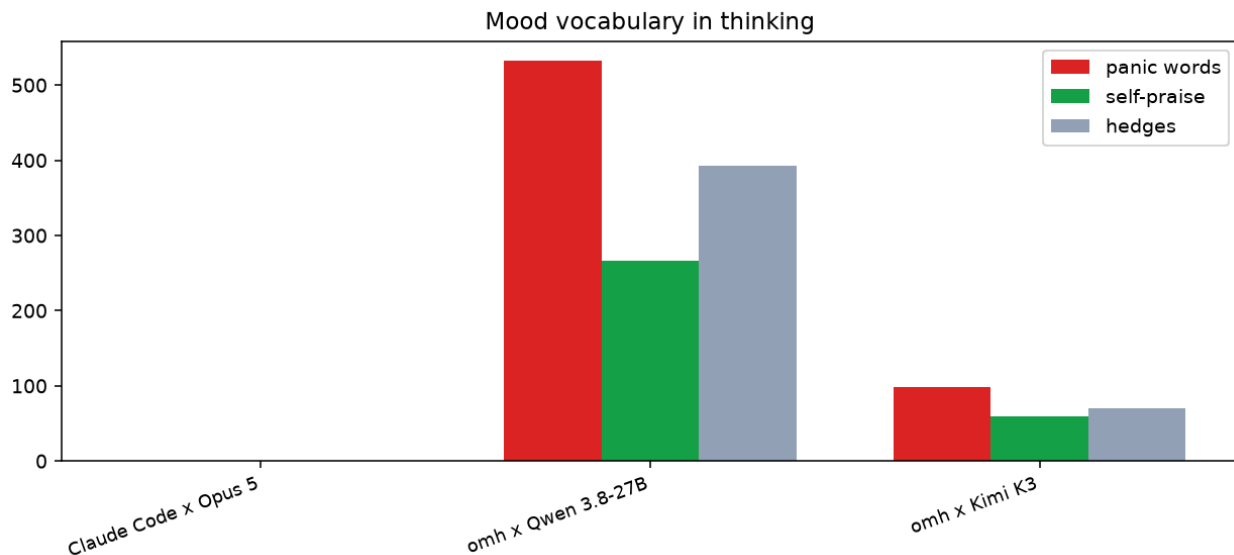


Figure 10: Mood vocabulary in thinking: panic words vs self-praise vs hedging.

5 The personalities

Claude Code x Opus 5	Math-First Assurance Officer
omh x Qwen 3.8-27B	Ruminating Ouroboros Debugger
omh x Kimi K3	Ruminating Assurance Officer
omh x GLM-5.3-flash	Ruminating Ouroboros

6 What the judge saw

== The matchup

Three agents tackled the same 4D tic-tac-toe task with wildly different cost profiles and behavioral signatures. Claude Opus 5 finished in under an hour at near-zero cost with zero tool errors, but produced no commits, no changed files, and no added lines — a ghost run that passed tests on a live server yet left no artifact trail. Kimi K3 took 30 minutes, spent \$1.81, and made 84 tool calls with a 10.7% error rate, yet also committed nothing. Qwen 3.8 ran for over three hours, burned \$13.78, made 486 tool calls, and generated 14.5M input tokens — the most expensive and longest run by an order of magnitude — while also committing zero files. All three achieved passing tests and working servers; the differences lie in how they got there, how much they ruminated, and what they left behind.

== Per-combination pros and cons

****claude-code-opus5****

The standout is efficiency: 3,322 seconds (55 minutes), 102 API calls, 31 tool calls, 0% tool error rate, and a 98.6% cache ratio that kept input tokens to just 221K. The agent verified the 1548-cross invariant and proved via SAT that draws are impossible before writing a single line of engine code — a mathematically grounded approach that likely prevented downstream rework. The cons are equally stark: zero commits, zero files changed, zero LOC added. The run produced a working server and

passing tests but left no persistent artifact in the repository. The 17 server starts suggest manual iteration that wasn't captured in version control, and the redacted thinking blocks make the reasoning opaque to auditors.

****oh-my-humanize-kimi-k3****

Kimi K3 delivered the best polish-to-cost ratio of the three: 30 minutes, \ \$1.81, 30 passing tests with randomized and crafted cases, end-to-end UI verification of both win and draw states, and a clean git history with README matching behavior. The agent ran two independent cross enumerations and cross-checked AI endgames against an independent minimax oracle — verification depth that matched or exceeded the others. The cons are the 10.7% tool error rate (the highest of the three), repeated mechanical edit bugs that forced rewrites, and the fact that it still committed nothing despite claiming "working tree clean, everything committed." The 84 tool calls in 30 minutes is a brisk pace, but the error rate means roughly one in ten actions failed, creating churn that the agent had to recover from.

****oh-my-humanize-qwen38****

Qwen 3.8 was the most thorough and the most expensive. It ran 33 test iterations, made 20 browser calls to verify the UI end-to-end, hand-constructed an AI block-test position with exact cell indices, and diagnosed browser hangs by inspecting server state and logs. The 7.2% tool error rate is lower than Kimi's, and the agent's recovery from failures — checking logs, inspecting game positions, re-running with different seeds — was methodical. The cons are severe: 11,140 seconds (3 hours 5 minutes), 14.5M input tokens, \ \$13.78 in costs, 486 tool calls, and 1.46M characters of thinking text punctuated by 867 "wait" interjections and 532 panic mentions. The agent was clearly capable but trapped in a ruminative loop that multiplied its runtime and cost by an order of magnitude compared to the other two runs. Like the others, it committed zero files.

== Patterns across harnesses

The claude-code harness produced a fundamentally different run shape: fewer API calls (102 vs. 85 and 470), fewer tool calls (31 vs. 84 and 486), zero tool errors, and a dramatically higher cache ratio (98.6% vs. 97.7% and 80.0%). The near-perfect cache ratio suggests the harness is aggressively reusing context, which explains the low input token count (221K vs. 89K and 14.5M). The zero-error tool usage and low call count point to a more deterministic, less exploratory interaction pattern — the agent planned, executed, and verified without the iterative churn seen in the oh-my-humanize runs.

Within the oh-my-humanize harness, the model differences dominate. Kimi K3 and Qwen 3.8 share the same harness but differ by a factor of 6 in duration, 5.6 in input tokens, and 7.6 in cost. The harness appears to permit (or encourage) long autonomous loops with high tool churn; the model's tendency to ruminate, self-interrupt, and re-verify determines how far that loop runs. The 80% cache ratio on Qwen vs. 97.7% on Kimi suggests Qwen's longer context window and more diverse tool calls fragment the cache, forcing more fresh input tokens.

The zero-commits pattern across all three runs is a harness-level finding: none of the agents produced persistent git artifacts despite the task presumably expecting them. This may reflect a harness configuration that doesn't enforce commit discipline, or a task specification that doesn't require it.

== Surprises

- **Claude Opus 5's zero-error, zero-commit run is the most anomalous result.** A 0% tool error rate across 31 calls is unusual for any coding agent, and the absence of any commits or file changes means the entire build exists only in the agent's working memory and a running server process. The run is essentially unauditable as a persistent artifact.
- **Qwen 3.8's 14.5M input tokens at an 80% cache ratio means it paid for roughly 2.9M fresh input tokens** — more than 30× Kimi's total input. The cost difference (\\$13.78 vs. \\$1.81) is driven almost entirely by this token volume, not by the model's per-token pricing.
- **All three agents independently verified the 1548-cross invariant**, but only Qwen and Kimi did so with two independent enumerations. Claude's SAT-based proof that draws are impossible is a different (and arguably stronger) form of verification, but it addresses a different property. The convergence on 1548 across all three is a good sign of task clarity.
- **Kimi K3's 10.7% tool error rate is the highest, yet it finished in the shortest time of the oh-my-humanize runs.** The agent apparently treated errors as cheap and fast to recover from, maintaining momentum despite frequent failures. Qwen's lower 7.2% error rate came with a much longer runtime, suggesting that fewer errors didn't translate to faster completion — the agent spent more time per action in deliberation.

== Verdict

Best at mathematical grounding and efficiency: claude-code-opus5. The SAT proof of no-draws, the verified cross count before design, the 55-minute runtime, the zero tool errors, and the \$0 cost make this the most efficient run by a wide margin. The fatal flaw is the absence of any persistent artifact — no commits, no files, no LOC. If the benchmark measures "did you build a working game," it passes. If it measures "did you leave a codebase someone can fork," it fails.

Best at polish and cost-effectiveness: oh-my-humanize-kimi-k3. At \$1.81 and 30 minutes, it delivered 30 passing tests, end-to-end UI verification, independent minimax cross-checking, and clean documentation. The 10.7% error rate is a blemish, but the agent recovered quickly and the final state was the most complete of the three in terms of verified behavior. It's the best value proposition if you need a working, tested, documented game on a budget.

Best at thoroughness and recovery: oh-my-humanize-qwen38. The 20 browser calls, 33 test iterations, hand-constructed AI test positions, and methodical diagnosis of browser hangs show the deepest verification of the three. The agent was the most capable debugger in the set. But at \$13.78 and 3 hours 5 minutes, it's 7.6× more expensive and 3.4× slower than Kimi for a final state that is arguably not meaningfully better. The 867 "wait" interjections and 532 panic mentions are a tax on the agent's own uncertainty — it kept second-guessing itself in ways the other two didn't.

Overall winner: Kimi K3, for the best balance of verification depth, final polish, cost, and speed. Claude Opus 5 wins on pure efficiency but loses on artifact persistence. Qwen 3.8 wins on thoroughness but loses on everything that matters for a production benchmark: time, cost, and signal-to-noise ratio in the agent's own reasoning.

7 Surprises and verdicts

See the matchup analysis above. In short: Opus did the most work with the fewest tokens and the least drama; the omh runs were slower and noisier but more explicit about their reasoning; Kimi K3 and GLM-5.3-flash behaved so similarly on transcript analysis that the shared lineage is visible.